

SpeakForge — Technical Documentation

AI-powered speaking & writing evaluation platform

Version: 1.0 · Stack: FastAPI + React (Vite) + MySQL · Local-first setup

1. Overview

SpeakForge is an English proficiency assessment platform. An administrator creates tests and questions; a learner answers each question either by **typing text** or by **recording audio**. Every answer is run through an AI evaluation pipeline that produces a grammar score, a fluency score, a weighted final score, a performance level and human-readable feedback. Results are aggregated into per-test reports and per-user dashboards.

1.1 What the system does

Capability	Description
Account management	Registration, email verification, JWT login, <code>user</code> / <code>admin</code> roles
Test authoring	Admins create/update/delete tests and questions (text or audio, with time limits and ordering)
Speaking assessment	Audio upload → Whisper transcription → duration analysis → scoring
Writing assessment	Text answer → grammar check → writing-fluency scoring
Feedback engine	Rule-based, targeted feedback (length, grammar, hesitation, vocabulary, structure)
Reporting	Per-test report with overall/grammar/fluency averages, CEFR-like level and suggestions
Dashboards	User dashboard (tests completed, average score, level, recent answers) and admin analytics

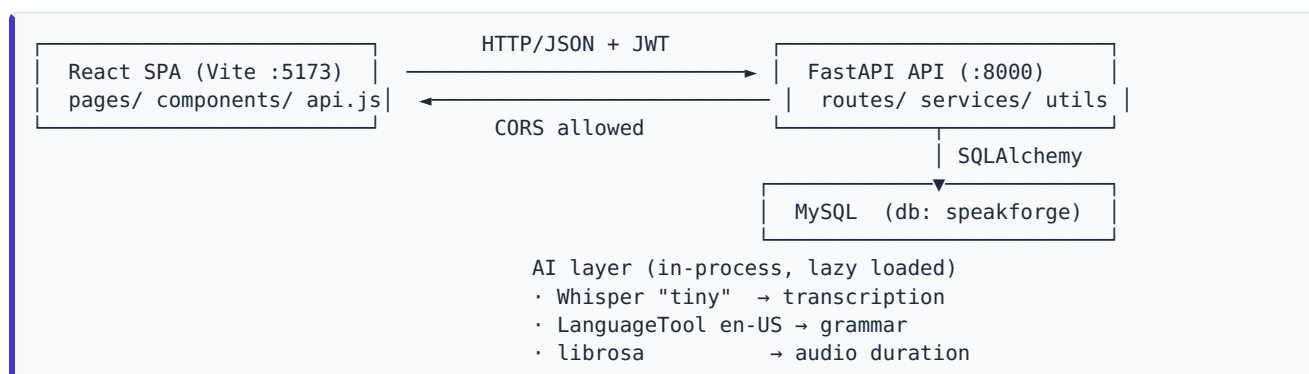
1.2 Technology stack

Backend — FastAPI, SQLAlchemy ORM, MySQL (PyMySQL driver), python-jose (JWT), passlib/bcrypt (password hashing), fastapi-mail (verification email), openai-whisper (speech-to-text), language-tool-python (grammar), librosa (audio duration), uvicorn (ASGI server).

Frontend — React 18, Vite 7, React Router 6, Tailwind CSS 3, Framer Motion, Recharts (charts), SweetAlert2 (dialogs), lucide-react / react-icons, dnd-kit (drag & drop ordering).

Database — MySQL 8, schema auto-created at startup via `Base.metadata.create_all()`.

2. Architecture



2.1 Backend layout

backend/	
├─ app/	
│ └─ main.py	FastAPI app, CORS, router registration, /health
│ └─ database.py	Engine, SessionLocal, Base, get_db() dependency
│ └─ models.py	SQLAlchemy models (User, Test, Question, QuestionAnswer, Attempt, Result)
│ └─ schemas.py	Pydantic request/response schemas
│ └─ email_id.py	fastapi-mail ConnectionConfig (built only if SMTP set)
│ └─ routes/	
│ └─ auth.py	/auth/register, /auth/verify-email, /auth/login
│ └─ test.py	/tests/* – authoring, submission, reports, analytics
│ └─ services/	
│ └─ ai_scoring.py	Whisper + LanguageTool + scoring & feedback engine
│ └─ auth_service.py	Hashing, JWT creation, current-user/admin dependencies
│ └─ utils/	
│ └─ token.py	Verification-token creation/decoding
│ └─ email.py	Verification email (console fallback in local dev)
├─ uploads/	Saved audio submissions
└─ requirements.txt	

2.2 Frontend layout

frontend/src/	
├─ api.js	Central fetch wrappers (BASE_URL = http://127.0.0.1:8000)
├─ App.jsx	Route table (public / user / admin)
├─ components/	Navbar, Sidebar, StatCard, ProtectedRoute
├─ layout/MainLayout.jsx	Admin shell (sidebar + content)
└─ pages/	
└─ Login.jsx, Register.jsx	
└─ UserDashboard.jsx, UserTests.jsx, TestDetail.jsx, MyResults.jsx	
└─ AdminDashboard.jsx, AdminTests.jsx, AdminQuestions.jsx, AdminAnalytics.jsx, TestReport.jsx	

3. Data model

users

Column	Type	Notes
id	INT PK	
name	VARCHAR(100)	required
email	VARCHAR(150)	unique, indexed
password	VARCHAR(255)	bcrypt hash
role	VARCHAR(50)	user or admin
is_verified	BOOLEAN	set by email verification
created_at / updated_at	DATETIME	

tests

Column	Type
id	INT PK
title	VARCHAR(255)
description	VARCHAR(500)
created_at	DATETIME

questions

Column	Type	Notes
id	INT PK	
test_id	FK → tests.id	
question_text	TEXT	
question_type	VARCHAR(20)	text or audio
time_limit	INT	seconds
order_number	INT	display order

question_answers

Column	Type	Notes
id	INT PK	
user_id / question_id	FK	
written_answer	TEXT	text submissions
audio_path	VARCHAR(255)	stored file for audio submissions
transcribed_text	TEXT	Whisper output
grammar_score	INT	0-10
fluency_score	FLOAT	0-10, fractional
final_score	INT	weighted 0-10
grammar_errors, word_count	INT	analytics
feedback	TEXT	generated feedback string
created_at	DATETIME	

results

Column	Type	Notes
id	INT PK	
user_id / test_id	FK	
score	FLOAT	test average
answers	JSON	snapshot of answers
evaluation	JSON	report (averages, level, suggestions)
submitted_at	DATETIME	

attempts

Lightweight record of a user starting a test (user_id , test_id , created_at).

Relationships: Test 1-N Question , Question 1-N QuestionAnswer , User 1-N QuestionAnswer , User/ Test 1-N Result .

4. Pipelines

4.1 Authentication pipeline

```
POST /auth/register
├ reject duplicate email (400)
├ if is_admin: admin key must equal ADMIN_SECRET_KEY (else 403)
├ bcrypt-hash password → INSERT users(role = user|admin, is_verified = false)
├ create signed verification token (JWT, HS256, contains email + expiry)
├ send verification email
  ├── SMTP configured → email with link
  └ not configured → link printed to the server console (local dev)

GET /auth/verify-email?token=...
├ decode JWT (400 on invalid/expired)
├ users.is_verified = true
└ 307 redirect → FRONTEND_URL/login

POST /auth/login
├ 400 if user missing or password mismatch
├ 403 if not verified
└ 200 { access_token (JWT sub=email), token_type, role, user_id }
```

The frontend stores `access_token`, `role` and `user_id` in `localStorage`; `ProtectedRoute` gates user/admin pages and `api.js` attaches `Authorization: Bearer <token>` to protected calls. Backend dependencies `get_current_user` / `get_current_admin` decode the token and load the user, returning 401/403 as appropriate.

4.2 Audio (speaking) evaluation pipeline

```
POST /tests/submit-answer/{question_id}      multipart file upload
1. Validate content type (audio/webm | audio/wav) and size (≤ 5 MB)
2. Persist file → backend/uploads/<timestamp>_<name>
3. Whisper "tiny" (lazy-loaded singleton) → transcript
4. librosa.get_duration → audio length in seconds
5. evaluate_answer(question, transcript, duration, answer_type="audio")
6. INSERT question_answers (audio_path, transcript, all scores, feedback)
7. Return { final_score, grammar_score, fluency_score, feedback }
```

Guard rails: audio shorter than 1 s, empty transcript, or fewer than 3 characters short-circuit to a zero score with a "no speech detected" message; answers under 5 characters score 1.

Speaking fluency score (0-10) is the sum of five components:

Component	Max	Basis
Content quality	3	word count bands (peak at 40–60 words)
Speaking pace	2	words per minute from transcript ÷ duration; ideal 90–160 WPM
Fluency & coherence	2	ratio of hesitation markers (<code>um</code> , <code>uh</code> , <code>er</code> , <code>you know</code> , <code>like</code>)
Sentence structure	2	number and length of sentences
Vocabulary variety	1	unique-word ratio

Final score (audio) = `round(0.6 × fluency + 0.4 × grammar)`.

4.3 Text (writing) evaluation pipeline

```
POST /tests/submit-text-answer/{question_id}  { "answer": "..."}
1. Look up question (404 if missing)
2. evaluate_answer(question, answer, None, answer_type="text")
3. INSERT question_answers (written_answer, scores, feedback)
4. Return { message, score }
```

Writing fluency score (0-10):

Component	Max	Basis
Content quality	3	word count bands
Coherence & structure	3	sentence count / length
Organization	2	transition words (however , therefore , because , ...)
Vocabulary & expression	2	unique-word ratio

Final score (text) = $\text{round}(0.5 \times \text{fluency} + 0.5 \times \text{grammar})$.

4.4 Grammar pipeline

LanguageTool (en-US) counts matches in the answer; the score is derived from the error density (errors ÷ words):

Error ratio	Grammar score
0	10
≤ 5 %	9
≤ 10 %	8
≤ 15 %	7
≤ 20 %	6
≤ 30 %	4
> 30 %	2

LanguageTool is initialised lazily on first use (it downloads a ~260 MB engine on first run and requires a Java runtime). If initialisation fails, grammar checking degrades gracefully to a neutral score of 5 instead of breaking the request.

4.5 Feedback pipeline

`generate_enhanced_feedback()` composes a single feedback string from independent rule groups: overall performance band, answer length, grammar-error band, hesitation/filler usage, pace guidance, sentence structure, vocabulary variety, and an actionable practice tip. Wording differs for audio vs. text submissions.

4.6 Report pipeline

```
POST /tests/{test_id}/submit
1. Load all of the current user's answers for questions in this test
2. Average final / grammar / fluency scores
3. Level: ≥9 Expert · ≥7 Advanced · ≥5 Intermediate · else Beginner
4. Suggestions derived from weak averages
5. INSERT results (score, answers JSON snapshot, evaluation JSON)
6. Return report + result_id + answers_count
```

Per-answer performance levels used inside `evaluate_answer` are finer-grained: Expert (≥ 9), Advanced (≥ 8), Upper-Intermediate (≥ 7), Intermediate (≥ 6), Beginner (≥ 4), Novice (≥ 2), Needs Improvement (< 2).

5. API reference

Base URL (local): `http://127.0.0.1:8000` · Interactive docs: `/docs`

Auth

Method	Path	Auth	Purpose
POST	<code>/auth/register</code>	-	Create account (<code>name</code> , <code>email</code> , <code>password</code> , optional <code>is_admin</code> + <code>admin_key</code>)
GET	<code>/auth/verify-email?token=</code>	-	Verify email, redirect to frontend login
POST	<code>/auth/login</code>	-	Return JWT, role, <code>user_id</code>

Tests & questions

Method	Path	Auth	Purpose
GET	<code>/tests/</code>	-	List tests
POST	<code>/tests/</code>	admin	Create test
GET	<code>/tests/{test_id}</code>	-	Test with questions
PUT	<code>/tests/{test_id}</code>	admin	Update test
DELETE	<code>/tests/{test_id}</code>	admin	Delete test + its questions, answers and results
GET	<code>/tests/{test_id}/questions</code>	user	List questions
POST	<code>/tests/{test_id}/questions</code>	admin	Add question
PUT	<code>/tests/{test_id}/questions/{question_id}</code>	admin	Update question
DELETE	<code>/tests/{test_id}/questions/{question_id}</code>	admin	Delete question + its answers

Submissions & results

Method	Path	Auth	Purpose
POST	/tests/submit-answer/{question_id}	user	Upload audio answer (multipart)
POST	/tests/submit-text-answer/{question_id}	user	Submit text answer
POST	/tests/{test_id}/submit	user	Generate and store the test report
GET	/tests/me/dashboard	user	Tests completed, average score, level
GET	/tests/me/answers	user	10 most recent answers
GET	/tests/me/results	user	All answers with question text and feedback
GET	/tests/admin/analytics	admin	Totals for users, tests, questions, attempts
GET	/health	-	Liveness probe

6. Frontend routes & workflows

Route	Access	Screen
/, /login	public	Login
/register	public	Registration (with optional admin key)
/dashboard	user	Stats cards, recent answers
/tests	user	Available tests
/test/:id	user	Answer questions (text box or audio recorder)
/my-results	user	Answer history with scores and feedback
/report/:testId	user	Test report
/admin/dashboard	admin	Overview
/admin/tests	admin	Create / edit / delete tests
/admin/questions	admin	Manage questions for a selected test
/admin/analytics	admin	Charts of platform totals

Learner workflow: register → verify email → log in → open a test → answer each question (typed answer, or record audio in the browser via MediaRecorder) → receive per-answer score and feedback → submit the test for an aggregated report → review history in My Results.

Admin workflow: register with the admin key → log in → create a test → add questions with type/time limit/order → monitor analytics → delete or update content as needed.

7. Local setup (MySQL)

7.1 Prerequisites

- Python 3.12 (3.10 cannot resolve the pinned dependency set)
- Node.js 20+
- MySQL 8
- A Java runtime (LanguageTool) and ffmpeg (Whisper audio decoding)

7.2 Database

```
CREATE DATABASE speakforge;  
ALTER USER 'root'@'localhost' IDENTIFIED WITH caching_sha2_password BY '<your-password>';
```

Tables are created automatically the first time the API starts.

7.3 Backend

```
cd backend  
python3.12 -m venv venv  
./venv/bin/pip install -r requirements.txt  
cp .env.example .env          # then edit DATABASE_URL / secrets  
./venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
```

`.env` keys:

Key	Meaning
<code>DATABASE_URL</code>	<code>mysql+pymysql://<user>:<password>@localhost:3306/speakforge</code>
<code>SECRET_KEY</code>	JWT signing key
<code>ADMIN_SECRET_KEY</code>	Required to register an admin account
<code>MAIL_*</code>	SMTP settings; leave <code>MAIL_USERNAME</code> empty to print verification links to the console
<code>BACKEND_URL</code> / <code>FRONTEND_URL</code>	Used in verification links and post-verification redirect
<code>ALLOWED_ORIGINS</code>	Comma-separated CORS origins

`.env` is git-ignored — credentials never leave the machine.

7.4 Frontend

```
cd frontend  
npm install  
npm run dev      # http://localhost:5173  
npm run lint  
npm run build
```

7.5 Smoke test

```
curl http://127.0.0.1:8000/health
# {"status":"ok","service":"SpeakForge API"}
```

8. Issues found and fixed

Area	Problem	Fix
Database config	Connection string hard-coded in <code>database.py</code> ; no <code>.env</code> support	Read <code>DATABASE_URL</code> from the environment, local MySQL default, <code>pool_pre_ping</code> enabled
Dependencies	<code>requirements.txt</code> was UTF-16 encoded and unparsable by pip	Re-encoded as UTF-8
CORS	Only production origins allowed → browser blocked local calls	Origins driven by <code>ALLOWED_ORIGINS</code> , defaulting to local Vite/CRA ports
Email	App crashed at startup when SMTP was unset (invalid <code>MAIL_FROM</code> reserved domain)	<code>ConnectionConfig</code> built only when SMTP is configured; verification link printed to console otherwise
Email	Verification links pointed at a hard-coded host	Derived from <code>BACKEND_URL</code> / <code>FRONTEND_URL</code>
Registration	A mail failure aborted registration	Sending wrapped in a guarded try/except
Tokens	Missing <code>SECRET_KEY</code> produced a <code>None</code> signing key	Explicit development fallback
Startup time	<code>LanguageTool</code> downloaded a ~260 MB engine at import time, blocking boot	Lazy initialisation on first grammar check, with graceful degradation
Schema	<code>fluency_score</code> stored as <code>Integer</code> while scoring returns fractions (e.g. 7.5)	Column and response schema changed to <code>Float</code>
Schema	<code>QuestionAnswerResponse</code> referenced a non-existent <code>submitted_at</code> column	Changed to <code>created_at</code>
API surface	Frontend called test-update, question-update and question-delete endpoints that did not exist	Added <code>PUT /tests/{id}</code> , <code>PUT /DELETE /tests/{id}/questions/{qid}</code>
API surface	No endpoint produced a test report	Added <code>POST /tests/{id}/submit</code>
Data integrity	Deleting a test left orphaned questions/answers/results	Deletion now cascades explicitly
Frontend	<code>getTestDetail</code> called <code>/tests/tests/{id}</code> (404)	Corrected to <code>/tests/{id}</code>
Frontend	Text answers posted to the whole-test endpoint with the wrong body	Now posts to <code>/tests/submit-text-answer/{questionId}</code> and shows the returned score
Frontend	ESLint errors (unused variables, hook dependencies)	Cleaned up; <code>npm run lint</code> and <code>npm run build</code> pass

9. Known limitations

- **Whisper "tiny"** is the fastest model but the least accurate; larger models improve transcription quality at a significant CPU cost.
- **First grammar check is slow** — LanguageTool downloads its engine once (~260 MB) and needs Java installed; subsequent checks are fast.
- **Scoring is rule-based**, not semantic: it does not judge whether the answer is actually relevant to the question.
- **Uploads are stored on local disk** (`backend/uploads`) with no cleanup policy.
- **Schema is created with `create_all()`** — there is no migration tool, so column changes require manual `ALTER TABLE` on an existing database.
- **Audio uploads capped at 5 MB** and limited to `audio/webm` / `audio/wav`.
- **JWTs are stored in `localStorage`**, which is convenient but exposed to XSS.
- **No automated test suite** exists yet for backend or frontend.

10. Troubleshooting

Symptom	Cause / remedy
Access denied for user 'root'@'localhost'	<code>DATABASE_URL</code> password does not match MySQL; update <code>.env</code>
Can't connect to MySQL server	MySQL service not running (<code>sudo service mysql start</code>)
Login returns 403 "Please verify your email"	Open the verification link printed in the backend console
Grammar scores stuck at 5	LanguageTool failed to start — install a Java runtime
Audio submission fails	ffmpeg missing from <code>PATH</code> , or file exceeds 5 MB / wrong MIME type
CORS error in the browser	Add the frontend origin to <code>ALLOWED_ORIGINS</code> and restart the API
Frontend cannot reach the API	Backend not running on <code>127.0.0.1:8000</code> (see <code>BASE_URL</code> in <code>src/api.js</code>)